

Polymarket Crypto Up/Down DQN Project

현재 진행 상황

구분	현재 상태
대상 asset	BTC/ETH/SOL/XRP 5m/15m/60m
주요 실험	DQN 계열 agent와 rule-based baseline 비교, time-lag와 performance 비교
action	hold, buy up, buy down
거래 단위	buy 1회 = Polymarket contract 1개 매수
reward	terminal settlement 기반 profit
평가 방식	time-series walk-forward 5-fold
주요 모델	TD(lambda) DQN, Double Dueling DQN, n-step Double Dueling DQN, PPO + GAE(lambda)

결과 요약

1. 목표는 Polymarket Up/Down market에서 agent가 매 minute마다 **hold, buy up, buy down** 중 하나를 선택하도록 학습시키는 것이다.
2. State는 Polymarket 가격 정보, Binance 1분 candle feature, 현재 portfolio 상태로 구성했고, reward는 market 종료 시 settlement profit을 기준으로 계산한다.
3. 5m/15m/60m 모두 TD(lambda) DQN과 Double Dueling DQN 계열을 비교하고, 60m에는 delayed reward 대응을 위해 PPO + GAE(lambda)를 추가했다.
4. 평가는 time-series walk-forward 5-fold로 진행한다. 최종 평균 결과는 5개 test fold의 out-of-sample 지표 평균이다.
5. Baseline은 대부분 episode 초반에 한 번만 매수하므로 DQN과 **mean_profit**을 직접 비교하면 exposure 차이가 섞인다. 따라서 **win_rate, avg_buy_up/down, hold_only_rate**를 함께 봐야 한다.
6. Time-lag 실험은 BTC no-lag, ETH Binance lag, SOL Polymarket lag, XRP Binance + Polymarket lag로 구성했다.
7. 실제 trading에서는 backtest보다 data/model latency가 중요하다. 따라서 time lag와 model accuracy 사이의 trade-off를 추가로 검증해야 한다.
8. 다음 단계는 state 변경, action 확장, reward shaping, model selection, hyperparameter tuning, robust training, rolling window 재학습, baseline 수정이다.

목차

1. 데이터와 state 구성
2. action 구성
3. reward와 profit 계산
4. DQN 모델 구성
5. baseline 모델 구성
6. 평가 방식과 test 상태
7. 결과값 해석
8. Agent 관점의 의사결정 과정
9. Time lag 실험 설계
10. 앞으로 더 생각해 볼 주제

1. 데이터와 State 구성

현재 model feature는 다음과 같이 구성되어 있습니다. Polymarket의 정보, Binance의 정보, 현재 Portfolio 상태

Polymarket, Binance 정보

feature	의미
<code>poly_yes_price</code>	Polymarket Up/YES contract 가격
<code>poly_overround</code>	$\text{yes} + \text{no} - 1$
<code>remaining_time_ratio</code>	남은 시간 비율
<code>btc_return</code>	Binance 1분 return
<code>btc_return_from_market_start</code>	market 시작 이후 누적 return
<code>btc_range_1m</code>	Binance 1분 high-low range
<code>btc_volume_log1p</code>	Binance 거래량 log scale
<code>btc_taker_buy_ratio</code>	Binance taker buy ratio
---	---

- ETH/SOL/XRP notebook에서는 `btc_` prefix가 각각 `eth_`, `sol_`, `xrp_`로 바뀝니다.
- Binance의 의미란 가상화폐의 원물의 정보를 의미합니다.
- `poly_overround`는 Yes 가격 + No 가격이 1에서 얼마나 벗어났는지 계산한 값입니다. 이론적으로 `poly_overround`는 0(즉, $\text{yes} + \text{no} = \1)가 나와야 하지만 polymarket 시장 상태에 따라 0이외의 값을 가질 수 있습니다.
- 결측치는 future leakage를 피하기 위해 전체 평균이 아닌 과거 expanding mean 값으로 대체했습니다.
- Binance와 polymarket data 모두 1분 간격의 data를 사용했습니다.
- Binance data의 출처입니다. (<https://data.binance.vision/?prefix=>) 1분 보다 더 작은 간격의 데이터를 이용할 수 있습니다.

Portfolio 상태

feature	의미
<code>cash_ratio</code>	현재 cash / initial cash
<code>portfolio_value_ratio</code>	현재 mark-to-market value / initial cash
<code>net_position_value_ratio</code>	up position value - down position value
<code>trades_remaining_ratio</code>	남은 trade 여력

- 해당 feature들은 environment가 runtime에 계산합니다.
- $\text{portfolio_value} = \text{cash} + \text{한 episode 내 up shares 수} * \text{현재 up 가격} + \text{한 episode 내 down shares 수} * \text{현재 down 가격}$ 입니다.
- `max_trade`를 5/15/60번으로 한정해서 남은 trade 여력일 파악해야 합니다.

이렇게 총 12개의 정보가 state를 구성합니다.

2. Action 구성

Action은 세 개입니다.

action id	action	의미
0	hold	아무것도 하지 않음
1	buy up	Up/YES contract 1개 매수
2	buy down	Down/NO contract 1개 매수
---	---	---

- buy action은 1달러어치 매수를 의미하지 않습니다.

예를 들어 YES 가격이 0.485이고 수수료가 0.2%라면:

```
contract cost = 0.485
fee = 0.485 * 0.002
total spend = 0.485 + 0.485*0.002 = 0.48597
up_shares += 1
```

- 정산 시에는 맞은 contract 1개당 1달러를 받습니다.
- buy action의 수량을 1개로 제한했는데, 추후 sell up, sell down가 추가되기 때문입니다.
- 또한, 연속적인 시장 상황에서 데이터의 입력은 이산적으로 이루어지기 때문에 해당 이산적인 횟수만큼을 최대 action의 합으로 정했습니다.

3. Reward와 Profit 계산

3.1 Settlement value

episode가 끝나면 market outcome에 따라 다음과 같이 정산됩니다.

```
outcome = 1: up share 1개당 1달러
outcome = 0: down share 1개당 1달러
```

정산 가치는 다음과 같이 정해집니다.

```
settlement_value = 남은 cash + 맞은 방향 share 수 * 1
```

3.2 Profit

결과표의 profit은 다음으로 계산된다.

```
profit = settlement_value - initial_cash
```

한 번의 buy up 예시는 다음과 같습니다.

상황	계산
YES 가격 0.48에 1개 매수	spend = 0.48 + fee
outcome이 Up	profit contribution $\approx 1 - \text{spend}$
outcome이 Down	profit contribution $\approx 0 - \text{spend}$
---	---

3.3 Reward scaling

학습용 reward는 profit을 그대로 쓰지 않고 $1 / \text{initial_cash}$ 로 scaling했습니다. DQN 학습에서 target 값이 지나치게 커지는 줄이기 위해서입니다.

4. DQN 모델 구성

사용한 강화학습 모델은 다음과 같습니다.

모델	역할
TD(lambda) DQN	eligibility trace로 terminal reward를 이전 action에 전파
Double Dueling DQN	replay buffer 기반 DQN, value와 advantage 분리
n-step Double Dueling DQN	15m/60m에서 reward 전달을 개선하기 위해 추가
PPO + GAE(lambda)	60m의 긴 horizon에서 actor-critic 방식으로 policy를 직접 학습

4.1 TD(lambda) DQN

TD(lambda)는 eligibility trace를 이용한다. terminal reward가 마지막에만 나오더라도 이전 step의 action들이 어느 정도 영향을 받게 됩니다.

```
trace <- gamma * lambda * trace + grad(Q(s,a))
parameter update <- TD error * trace
```

5m처럼 짧은 episode에서는 효과가 제한적일 수 있지만, 15m/60m처럼 episode가 길어질수록 의미가 커집니다.

4.2 Dueling DQN

Dueling DQN은 Q값을 바로 예측하지 않고 다음 두 부분으로 나눕니다.

$$Q(s, a) = V(s) + A(s, a)$$

항목	의미
$V(s)$	현재 state 자체가 좋은지
$A(s, a)$	그 state에서 특정 action이 얼마나 더 좋은지

Polymarket에서는 어떤 상태에서 action 간 Q-value의 차이가 작을 수 있습니다. 이때 state value($V(s)$)와 action advantage($A(s, a)$)를 분리하면 안정된 학습을 할 수 있습니다.

4.3 Double DQN

DQN은 Q-value의 값을 overestimate하는 경향이 있는데, Q-valueDouble DQN은 action 선택과 target Q 평가를 분리해 Q-value overestimation을 줄입니다.

```
next_action = argmax Q_online(next_state)
target Q = Q_target(next_state, next_action)
```

4.4 Double Dueling DQN

Double Dueling DQN은 Double DQN과 Dueling network를 합친 모델입니다.

구성요소	역할
Dueling network	$Q(s, a)$ 를 $V(s)$ 와 $A(s, a)$ 로 나누어 표현
Double DQN target	다음 action 선택과 target Q 평가를 분리
---	---

즉 현재 state에서 action value를 계산할 때는 Dueling 구조를 사용하고, 학습 target을 만들 때는 Double DQN 방식을 사용합니다.

```
Q_online(s, a) = V_online(s) + A_online(s, a)

next_action = argmax_a Q_online(next_state, a)
target Q = Q_target(next_state, next_action)
```

따라서, 다음 두 가지의 효과를 낼 수 있습니다.

1. action 간 Q값 차이가 작을 때 state value와 action advantage를 분리
2. max 연산 때문에 Q값이 과대평가되는 문제를 줄임

4.5 n-step Double Dueling DQN

n-step Double Dueling DQN은 Double Dueling DQN에 n-step return을 추가한 것입니다. 즉 네트워크 구조는 Dueling이고, target 계산은 Double DQN이며, reward target은 1-step이 아니라 n-step으로 만드는 복합적인 모델입니다.

```
n-step target =
r_t + gamma r_{t+1} + ... + gamma^{n-1} r_{t+n-1}
+ gamma^n Q_target(s_{t+n}, argmax_a Q_online(s_{t+n}, a))
```

15m/60m에서는 reward가 market 종료 시점에 발생하기 때문에, 1-step update만 사용하면 초반 action까지 reward가 전달되지 않습니다. n-step return은 몇 step 뒤의 reward를 한 번에 target에 포함해 delayed reward 문제를 줄일 수 있습니다.

5m/15m/60m 모델에 다음과 같이 적용했습니다.

horizon	사용 방식
5m	Double Dueling DQN 중심
15m	5-step Double Dueling DQN
60m	10-step Double Dueling DQN
---	---

4.6 PPO + GAE(lambda)

60m에서는 한 episode 안에서 decision point가 60개라서 마지막 settlement reward가 매우 늦게 도착합니다. 이를 개선하기 위해 60m에는 Actor-Critic 계열인 PPO + GAE(lambda)를 추가했습니다.

Actor-Critic은 Q-value를 직접 고르는 DQN과 달리, policy를 출력하는 actor와 state value를 추정하는 critic을 함께 학습합니다.

```
Actor: pi(a | s), 지금 state에서 어떤 action을 할지 결정
Critic: V(s), 지금 state가 얼마나 좋은 상태인지 평가
```

PPO는 Actor가 policy를 너무 급격하게 바꿔 학습이 불안정해지는 것을 예방합니다. Policy의 변화량을 제한해서 (clipping) 이를 구현합니다.

GAE(lambda)(Generalized Advantage Estimation)는 critic의 value estimate를 이용해 각 step의 advantage를 계산합니다. 직관적으로는 마지막 reward를 모든 이전 action에 똑같이 나누는 것이 아니라, 시간 차이와 value estimate를 고려해서 어느 action이 더 좋았는지 부드럽게 배분하는 방식입니다. TD(lambda)와 비슷한 방법이라고 생각하시면 됩니다. 다만, TD(lambda)와 다르게 action이 평균 기대보다 어느 정도 좋았는지 actor에게 Advantage를 알려주는 과정입니다. 여기서 Advantage = 실제로 얻은 결과 - Critic이 예상한 결과입니다. 이를 통해, Action이 기대보다 좋았는지 나빴는지 확인할 수 있습니다.

```
advantage_t = GAE(reward_t, V(s_t), V(s_{t+1}), gamma, lambda)
```


현재 PPO는 60m 실험에서만 추가 비교 모델로 사용했습니다.

항목	현재 설정
gamma	0.995
GAE lambda	0.95
rollout episodes	16
update epochs	4
clip eps	0.2
entropy coef	0.01
---	---

PPO + GAE는 기존 DQN을 대체하기보다는, 60m처럼 delayed reward가 큰 환경에서 DQN 계열보다 policy 학습이 안정적인지 비교하기 위한 모델입니다.

5. Baseline 모델 구성

baseline으로 설정한 모델들은 다음과 같습니다.

baseline	설명
random	가능한 action 중 무작위 선택
buy_yes_t0	첫 시점에 Up 1개 매수 후 hold
buy_no_t0	첫 시점에 Down 1개 매수 후 hold
market_price_t0	첫 시점에 Polymarket 가격이 더 높은 쪽 매수
btc_momentum_t0	첫 시점에 Binance return 방향을 따라 매수
---	---

market_price_t0는 다음과 같은 규칙입니다.

```
poly_yes_price >= poly_no_price -> buy up
else -> buy down
```

btc_momentum_t0는 다음과 같은 규칙입니다.

```
btc_return >= 0 -> buy up
else -> buy down
```

여기서 `btc_return`의 의미는 Binance의 1분 candle 기준 수익률 입니다.(현재 1분 candle close / 이전 1분 candle close) - 1

-baseline에서 mean profit은 unfair하게 설계되어 있습니다.(episode에서 구매 횟수가 1) 따라서, 승리 확률로 확인하시면 될 것 같습니다.

6. 평가 방식과 Test 상태

6.1 Walk-forward 5-fold

평가는 random shuffle이 아니라 time-series walk-forward 방식으로 구성했습니다.

5m fold의 예시입니다.

```
fold 1: 2/19 ~ 3/29 -> 3/29 ~ 4/5
fold 2: 2/19 ~ 4/5 -> 4/5 ~ 4/12
fold 3: 2/19 ~ 4/12 -> 4/12 ~ 4/20
fold 4: 2/19 ~ 4/20 -> 4/20 ~ 4/26
fold 5: 2/19 ~ 4/26 -> 4/26 ~ 4/30
```

6.2 Training log와 final result의 차이

각 ipynb 파일의 8번 chunk의 학습 중 출력되는 profit은 train set에서 최근 `LOG_EVERY` episode의 평균입니다.

```
episode: 4000 출력 profit = 대략 episode 3501~4000 평균 profit
eps: eps-greedy의 확률
```

training 시에는 epsilon-greedy, test 시에는 greedy action selection을 진행했습니다.

7. 결과값 해석

결과표의 주요 열은 다음과 같습니다.

열	의미
<code>mean_profit</code>	episode 하나당 평균 profit
<code>median_profit</code>	episode profit 중앙값
<code>total_profit</code>	test set 전체 profit 합
<code>std_profit</code>	episode profit 표준편차
<code>win_rate</code>	profit > 0인 episode 비율
<code>sharpe_like</code>	mean_profit / std_profit

명	의미
avg_settlement_value	종료 후 평균 자산 가치
avg_buy_up	episode당 평균 buy up 횟수
avg_buy_down	episode당 평균 buy down 횟수
hold_only_rate	한 번도 매수하지 않은 episode 비율

avg_buy_up + avg_buy_down + hold_only_rate는 5가 되지 않습니다. 이유는 hold_only_rate는 episode 구간에서 buy를 전혀 하지 않은 비율이기 때문입니다.

결과를 해석할 때 다음 사항들을 고려할 수 있습니다.

1. mean_profit이 0보다 큰가?
2. sharpe_like가 baseline보다 높은가?
3. win_rate와 mean_profit이 함께 개선되는가?
4. hold_only_rate가 너무 높아 agent가 사실상 아무것도 하지 않는가?
5. avg_buy_up/down이 한쪽으로 과도하게 쏠리는가?

8. Agent 관점의 의사결정 과정

아래는 agent가 한 episode 안에서 실제로 의사결정하는 흐름입니다.

1. market 시작
2. initial_cash = 100
3. minute_idx = 0 state 관찰
4. Q-network가 hold / buy_up / buy_down Q값 계산
5. 학습 중이면 epsilon-greedy로 탐색 가능
6. action 선택
7. buy라면 현재 Polymarket 체결 가격으로 contract 1개 매수
8. 다음 minute으로 이동
9. market 종료까지 반복
10. outcome에 따라 settlement
11. profit 계산
12. DQN update

9. Time Lag 실험 설계

현재 5m/15m/60m 자산별 lag 실험은 다음처럼 구성되어 있다.

asset	Binance lag	Polymarket lag	목적
BTC	없음	없음	no-lag 기준군
ETH	있음	없음	Binance 지연 영향
SOL	없음	있음	Polymarket 지연 영향

asset	Binance lag	Polymarket lag	목적
XRP	있음	있음	양쪽 지연 동시 영향

agent가 보는 poly_yes_price는 lagged
실제 체결 가격 exec_poly_yes_price는 current-row

이렇게 한 이유는 실제 trading에서 가격 정보가 늦게 들어오더라도, 주문이 체결되는 가격은 현재 시장 가격이기 때문입니다.

10. 개선사항

10.1 State 변경

추가 후보:

후보	기대 효과
Polymarket price change	시장 내부 momentum 반영
spread / overround 변화	friction 변화 반영
Binance rolling volatility or LSTM	단기 변동성 regime 반영
cumulative volume ratio	거래량 폭증 감지
time-to-close nonlinear feature	종료 직전 행동 변화 반영
---	---

10.2 Action 추가

현재는 contract 1개 buy만 가능합니다. Sell을 고려할 수 있고, buy/sell의 규모 또한 고려할 수 있습니다.

10.3 Reward 구성

현재는 terminal reward 중심입니다. 추가 가능한 후보는 다음과 같습니다.

reward 방식	장점	단점
terminal reward	실제 settlement와 일치	reward가 늦게 옴
mark-to-market	중간 학습 신호가 많음	noise가 커질 수 있음
terminal + risk penalty	과도한 exposure 억제	penalty 설계 필요
profit per capital used	자본 효율 반영	trade 빈도 해석 필요

10.4 DQN 모델 selection

또 다른 DQN 모델 혹은, Actor-Critic 모델들을 고려할 수 있습니다.

10.5 Time lag - model accuracy tradeoff

실제 trading에서는 Time lag - model accuracy tradeoff가 있습니다. Time lag를 고려하지 않은 BTC가 가장 성능이 좋았고, time-lag를 고려하면 baseline 승률보다 낮게 나왔습니다. Time lag를 1m으로 잡아서(Data의 입력 단위) 극단적으로 결과가 안좋아졌다고 생각합니다. Binance data는 굉장히 1s 이하도 존재합니다. (Polymarket data는 1m 보다 작은 data가 있는지 확인해봐야 할 것 같습니다.)

빠른 결정: 최신 시장을 놓치지 않지만 feature/model 산출이 불안정할 수 있음
 느린 결정: feature는 안정적이지만 edge가 사라질 수 있음

10.6 Hyperparameter tuning

우선 조정할 값:

항목	5m	15m	60m
gamma	0.95	0.99	0.995
lambda	0.85	0.92	0.95
n-step	1	5	10
max trades	없음/5	15	20
episodes	5000	4000	2500

추가로 learning rate, hidden dimension, replay warmup, epsilon decay도 tuning 대상입니다.

10.7 Robust Training

이전에 말씀하셨던 학습 과정에서 noise를 추가해서 robust한 모델을 만드는 방법도 고려할 수 있습니다.

10.8 Rolling Window 재학습

새로운 정보들이 계속 들어오면서 모델을 재학습해야할 필요가 생깁니다.

10.9 Baseline 수정

Baseline의 profit이 unfair해서 DQN 모델과 fair한 비교를 위한 수정이 필요합니다.